

E-commerce REST API

Endpoint Reference — Node.js (Express) + PostgreSQL

Base URL	http://localhost:4000/api/v1
Auth header	Authorization: Bearer <token>
Health	GET http://localhost:4000/health
Response	success: { "success": true, "data": ... } error: { "success": false, "error": {...} }
Demo logins	admin@example.com / Admin123! customer@example.com / Customer123!

Authentication

METHOD	PATH	AUTH	BODY / PARAMS
POST	/auth/signup	Public	{ name, email, password }
POST	/auth/login	Public	{ email, password }
POST	/auth/forgot-password	Public	{ email } — reset token logged to server console
POST	/auth/reset-password	Public	{ token, password }

Products

METHOD	PATH	AUTH	BODY / PARAMS
GET	/products	Public	query: page, limit, category, minPrice, maxPrice, q, sort (created_at price name), order (asc desc)
GET	/products/:id	Public	—
POST	/products	Admin	{ name, price, category, stock?, description?, image_url? }
PUT	/products/:id	Admin	any subset of product fields
DELETE	/products/:id	Admin	—

Wishlist (protected)

METHOD	PATH	AUTH	BODY / PARAMS
GET	/wishlist	User	—
POST	/wishlist	User	{ productId }
DELETE	/wishlist/:productId	User	—

Shopping Cart (protected)

METHOD	PATH	AUTH	BODY / PARAMS
GET	/cart	User	returns line items + dynamically calculated total
POST	/cart	User	{ productId, quantity } — increments if already in cart
PATCH	/cart/:productId	User	{ quantity } — 0 removes the line
DELETE	/cart/:productId	User	—

Checkout & Orders (protected)

METHOD	PATH	AUTH	BODY / PARAMS
POST	/orders/checkout	User	validates live stock, creates a PENDING order
POST	/orders/:id/confirm-payment	User	{ success: true } != PAID, stock reduced, cart cleared
GET	/orders	User	query: page, limit
GET	/orders/:id	User	own order detail (ownership enforced)

Typical flow

1. POST /auth/login — capture the JWT token.
2. GET /products — browse and find product IDs.
3. POST /cart — add items (quantity increments if already present).
4. GET /cart — review the dynamically calculated total.
5. POST /orders/checkout — stock validated, a PENDING order is created.
6. POST /orders/:id/confirm-payment — order becomes PAID, stock reduced, cart cleared.
7. (Admin) POST /products — manage catalogue after logging in as admin.

Import postman_collection.json into Postman, or use requests.http in VS Code, to exercise these endpoints.